

Tema 2 (Part 2) – Estructura d'un programa

Secció 2.4

Assignatura: Introducció a la programació

Grau Enginyeria Informàtica

Curs 26-27

Inmaculada Rodriguez Santiago

Departament de Matemàtiques i Informàtica

Contingut

2.4 Estructures de control avançat

Disseny sistemàtic de estructures de control iteratives

Concepte de seqüència

Esquemes de recorregut i cerca



Contingut

2.4 Estructures de control: iteratives (avançat)

2.4.1 Introducció

2.4.2 Concepte de seqüència

2.4.3 Esquemes de programació de seqüències

2.4.4 Esquema de recorregut

2.4.5 Esquema de cerca

2.4.6 Recomanacions

Encara que les estructures de control iteratives (for, while) són estructures bàsiques, podem definir-les seguint un mètode sistemàtic. Això ens ajudarà a definir correctament les iteratives i evitar errors.

Es parla de **sentència iterativa** quan s'ha de repetir l'execució d'una sentència/es segons si es compleix una condició booleana.

while

```
Si (Sentències inicials)
while ( condició_booleana ) {
    <S ( Sentències)>
}
Sf (Sentències finals)
```

Correctesa: S'han de garantir l'estat inicial abans del `while`, la condició booleana de control del `while` i les sentències internes per a que la sentència iterativa acabi en algun moment.

- Inicialment s'executen les sentències S_i , després s'avalua l'expressió booleana del `while`. Si és certa s'executen les sentències S . Es torna a analitzar la condició després d'executar S .
- Les accions del cos del `while` (S) només s'executen si la condició booleana és certa.
- Quan la condició booleana és falsa es passen a executar les sentències finals de després del `while` (S_f)

2.4.1 Introducció

Exemples d'iteratives simples

```
int num;

num = 1;
while (num <= 100) {
    printf(" %d ", num);
    num = num + 1;
}
```

```
int num = 0;
/* i amb num = 3? i amb num = -1?
   i amb num = 4?
*/
while (num != 0) {
    num = num - 2;
    printf("num: %d\n", num);
}
printf("Fi del programa");
```

2.4.1 Introducció

Exemples d'iteratives simples

```
int num;  
  
num = 1;  
while (num <= 100) {  
    printf("%d ", num);  
}
```

2.4.1 Introducció

for

```
Si (Sentències inicials)
for ( int idx=0; idx < n ; idx++ ) {
    <S ( Sentències)>
}
Sf (Sentències finals)
```

- Quan s'ha de repetir l'execució d'una sentència/es n vegades

Correctesa: S'han de garantir l'estat inicial abans del for, la condició booleana de control del for i les sentències internes per a que la sentència iterativa acabi en algun moment.

```
int num = 2;
for (int i = 1; i <= 10; i++)
    printf(num + " x " + i + " =" + num*i);
```

Sortida a pantalla:

2	x	1	=	2
2	x	2	=	4
2	x	3	=	6
2	x	4	=	8
2	x	5	=	10
2	x	6	=	12
2	x	7	=	14
2	x	8	=	16
2	x	9	=	18
2	x	10	=	20

2.4.1 Introducció

Iteratives aniuades

Per cada iteració del bucle extern, s'executen totes les iteracions de l'intern.

```
for (int i = 1; i <= 10; i++) {  
    printf("Tabla de multiplicar del %d\n", i);  
    for (int j = 1; j <= 10; j++)  
        printf("%d x %d = %d\n", i, j, i*j);  
}
```

Sortida a pantalla:

Totes les taules de multiplicar,
des de 1 al 10

do while

```
Si (Sentències inicials)
do {
    < S (Sentències) >
} while (condició_booleana);
Sf (Sentències finals)
```

Correctesa: S'han de garantir l'estat inicial abans del `do while`, la condició booleana de control del `do while` i les sentències internes per a que la sentència iterativa acabi en algun moment.

- Iterativa on la condició es comprova al final de cada iteració! Per tant, la iterativa s'executa al menys una vegada.
- A continuació parlarem de esquemes de recorregut i cerca. Aquests esquemes es basen només en `while` i `for`. Per tant, **només utilitzaria `do ... while` en casos excepcionals en els que hi ha una millora en la legibilitat del codi.**

2.4.1 Introducció

OBO (Off by One error)

Error que es produeix quan en una iterativa iterem un nombre de vegades incorrecte (més o menys vegades). Per exemple, quan utilitzem $\leq$ en una condició i hauria de ser $<$, o comença la seqüència en 1 quan hauria de començar en 0.

Exemple. Volem executar la iterativa 10 vegades

OBO

```
i = 1;
while (i < 10) {
    ...
    i++;
}
```

OBO

```
i = 0;
while (i <= 10) {
    ...
    i++;
}
```



```
i = 1;
while (i <= 10) {
    ...
    i++;
}
```



```
i = 0;
while (i < 10) {
    ...
    i++;
}
```

A continuació tractarem com definir seqüències i condicions de parada de les iteratives, així com diferenciar entre recorregut i cerca.

Tot això, ajudarà a definir correctament les iteratives i evitar errors OBO.

Contingut

2.4 Estructures de control: iteratives (avançat)

2.4.1 Introducció

2.4.2 Concepte de seqüència

2.4.3 Esquemes de programació de seqüències

2.4.4 Esquema de recorregut

2.4.5 Esquema de cerca

2.4.6 Recomanacions

2.4.2 Concepte de seqüència

14

- La construcció més complicada d'usar és la composició iterativa. Cal un mètode per dissenyar iteracions de manera que:
 - El disseny aconseguit sigui correcte.
 - Sigi un mètode universal i senzill.
- Passos a realitzar.
 1. Caracterització de la seqüència (Identificació, Primer, Següent, FinalSeq).
 2. Identificació de l'esquema a aplicar: recorregut (R) o cerca (C).
 3. Aplicació de l'esquema (segons un model segons sigui R o C).
 4. Comprovació de casos extrems (tractament del darrer element, tractament de la seqüència buida).



A partir de la pràctica P2 s'han de caracteritzar les seqüències en cada iterativa, i identificar l'esquema R o C.

2.4.2 Concepte de seqüència

- Podem incloure una sentència iterativa dins del bloc de sentències d'una altra sentència iterativa.
- En aquests casos s'analitzen les composicions iteratives per separat (es poden tenir recorreguts dins de recorreguts, cerques dins de recorreguts, etc.)
- Es realitza el mateix procés d'identificar la seqüència i identificar l'esquema en cada bucle implicat en la solució.
- Exemple: Llistar per pantalla les taules de multiplicar de 1 al 10.

2.4.2 Concepte de seqüència

- Una seqüència és conjunt finit d'elements.
- La caracterització d'una seqüència es fa indicant:

Identificació de la seqüència: és una descripció que indica quins elements formen part de la seqüència.

Primer (): S_0 //Primer element de la seqüència.

Següent (S_i): S_{i+1} //Regla per a calcular el següent element.

FinalSeq (S_i): (condició_booleà) //Regla o propietat per saber si podem continuar avançant en la seqüència. Identifica el primer element que NO és de la seqüència.

2.4.2 Concepte de seqüència

17

- Una seqüència és conjunt finit d'elements.
- La caracterització d'una seqüència es fa indicant:

Identificació de la seqüència: és una descripció que indica quins elements formen part de la seqüència.

Primer (): S_0 //Primer element de la seqüència.

Següent (S_i): S_{i+1} //Regla per a calcular el següent element.

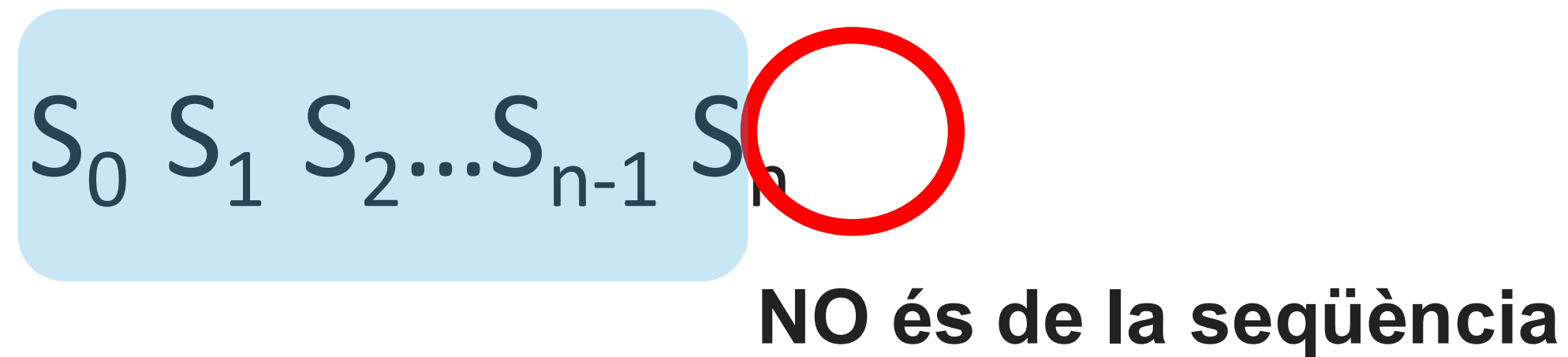
FinalSeq (S_i): (condició_booleà) //Regla o propietat per saber si podem continuar avançant en la seqüència. Identifica el primer element que NO és de la seqüència.

2.4.2 Concepte de seqüència

FinalSeq (S_i): (condició_booleà) //Regla o propietat per saber si podem continuar avançant en la seqüència. Identifica el primer element que NO és de la seqüència (circle vermell). 2 situacions:

- Es coneixen el nombre d'elements de la seqüència.

Seqüència de n elements:



- El primer element que no és de la seqüència (o **sentinella**) es pot identificar.

Seqüència fins sentinella:



2.4.2 Concepte de seqüència

Quina és la seqüència?

1. Identificació de la seqüència:
2. Primer ():
3. Següent (x):
4. FinalSeq (x):

PLANTILLA A
EMPLENAR PER A
IDENTIFICAR LA
SEQÜÈNCIA

2.4.2 Concepte de seqüència

Quina és la seqüència?

► **Exemple 1: caracterització de la seqüència [1 - 100]**

1. Identificació de la seqüència:

2. Primer ():

3. Següent ($\times$):

4. FinalSeq ($\times$):

2.4.2 Concepte de seqüència

Quina és la seqüència?

► **Exemple 1: caracterització de la seqüència [1 - 100]:**

1. Identificació de la seqüència: enters de l'1 al 100

2. Primer (): 1

3. Següent (x): $x+1$

4. FinalSeq (x): $(x > 100)$

Seqüència d'elements:

1 2 3 ... 100 101

Quina és la seqüència?

► **Exemple 2: caracterització de la seqüència ['a' 'e' 'i' 'o' 'u']:**

1. Identificació de la seqüència:

2. Primer ():

3. Següent (x):

4. FinalSeq (x):

Quina és la seqüència?

► **Exemple 2: caracterització de la seqüència ['a' 'e' 'i' 'o' 'u']:**

1. Identificació de la seqüència: vocals de la 'a' a la 'u'

2. Primer (): 'a'

3. Següent (x): `switch (x) {case 'a': seg='e'; break;`
`case 'e': seg='i'; break; ... }`

4. FinalSeq (x): `! (x == 'a' || x == 'e' || x == 'i' || x == 'o' || x == 'u' )`

Seqüència d'elements:

'a' 'e' ... 'u' 'z'

Quina és la seqüència?

► **Exemple 3: usuari entra comandes fins que indica “sortir”:**

1. Identificació de la seqüència:

2. Primer ():

3. Següent ($\times$):

4. FinalSeq ($\times$):

Quina és la seqüència?

► Exemple 3: **usuari entra comandes fins que indica “sortir”:**

1. Identificació de la seqüència: comandes entrats per teclat

2. Primer (): `scanf ("%s" , x) ;` *// %s indica que x es una cadena*

3. Següent (x): `scanf ("%s" , x) ;`

4. FinalSeq (x): `strcmp (x, "sortir") == 0` *// strcmp compara cadenas, #include <string.h>*

2.4.2 Concepte de seqüència

Quina és la seqüència?

- ▶ **Exemple 4: Si el jugador vol jugar una altra partida ho indica amb el caràcter 's', en cas contrari ho indica amb 'n'**

1. Identificació de la seqüència:

2. Primer ():

3. Següent (x):

4. FinalSeq (x):

2.4.2 Concepte de seqüència

27

Quina és la seqüència?

- ▶ **Exemple 4: Si el jugador vol jugar una altra partida ho indica amb el caràcter 's', en cas contrari ho indica amb 'n'**

1. Identificació de la seqüència: entrades per teclat ('s', 'n') per indicar si vol jugar una altra partida

2. Primer (): `'s'`

3. Següent (x): `scanf ("%c", x)`

4. FinalSeq (x): `x == 'n'`

Amb ' ' indico literal caràcter, més endavant veurem strings/cadenes i s'indicaràn amb " "

Seqüència d'elements:

`'s' 's' ... 's' 'n'`

Final de la seqüència

```
#include <stdio.h>

int main() {
    char cmd;
    /* Identificació de la seqüència: Seqüència de
       Strings/Comandes entrats pel teclat.
       Primer()= scanf("%c", &cmd)
       Següent(cmd)= scanf("%c", &cmd)
       FinalSeq(cmd)= cmd == 's'
       Esquema: recorregut
    */
    printf("Comanda?\n");
    scanf(" %c", &cmd);
    while (cmd != 's') {
        printf("Processant comanda %c\n", cmd);
        printf("Comanda?\n");
        scanf(" %c", &cmd);
    }
    return 0;
}
```

Contingut

2.4 Estructures de control: iteratives (avançat)

2.4.1 Introducció

2.4.2 Concepte de seqüència

2.4.3 Esquemes de programació de seqüències

2.4.4 Esquema de recorregut

2.4.5 Esquema de cerca

2.4.6 Recomanacions

2.4.3 Esquemes de programació de seqüències

Esquema de programació: patró de solució que es pot aplicar per a resoldre un ventall de problemes concrets.

Esquemes de programació de seqüències:

- de **recorregut**: per a recórrer tots els elements de la seqüència.
- de **cerca**: per a recórrer la seqüència fins a trobar un determinat element o fins a que es compleixi una certa condició booleana.

Contingut

2.4 Estructures de control: iteratives (avançat)

2.4.1 Introducció

2.4.2 Concepte de seqüència

2.4.3 Esquemes de programació de seqüències

2.4.4 Esquema de recorregut

2.4.5 Esquema de cerca

2.4.6 Recomanacions

2.4.4 Esquema de recorregut

32

Objectiu: Donada una seqüència d'elements es vol accedir a **TOTS ells** per a aplicar-los un cert tractament.

Esquema/Model:

```
<inicialitzacions>
elem = Primer();
while ( !FinalSeq(elem) ) {
    <tractar_element>
    elem = Següent(elem);
}
<finalitzacions> //Tractament de l'últim element
```

Veure com NO hi
ha break ni
continue!!

2.4.4 Esquema de recorregut

33

L'esquema de recorregut garanteix que:

- Cada element de la seqüència només es tracta *una vegada*.
- Sempre es tracta l'element actual primer i després *avança* al següent element.
- No es tracta cap element que *no sigui* de la seqüència.
- A cada iteració *decreix* la part dreta de la seqüència.
- En acabar, *tots* els elements de la seqüència s'han tractat (la part dreta de la seqüència és buida).

Seqüència de n elements:

$S_0 S_1 S_2 \dots S_{n-1} S_n$

NO és de la seqüència

Seqüència fins sentinella:

$S_0 S_1 S_2 \dots$ Sentinella

NO és de la seqüència

2.4.4 Esquema de recorregut

Exemple: Calcular la mitjana dels quadrats dels 100 primers naturals.

- 1. Identificació de la seqüència: seqüència d'enters [1...100].

Primer() = 1, Següent (x) = x +1, FinalSeq (x) = (x > 100)

- 2. Identificació de l'esquema: Recorregut

```
int    x;
int    suma;
float  mitjana;
suma = 0;                                // Inicialitzacions
x = 1;                                   // Primer Element
while (x <= 100) {                       // while (!FinalSeq)
    suma = suma + x*x;                  // Tractament
    x = x + 1;                         // Seguent Element
}
mitjana = suma / 100.0f;
printf ("La mitjana és %d\n", mitjana);
```

Exercici



ratxa.c: Escriu un programa que llegeixi una seqüència de nombres enters acabada en 0 i mostri la longitud de la ratxa més llarga de nombres consecutius iguals.

Exemple: per a l'entrada 5 2 7 7 7 7 0 la sortida ha de ser 4.

- 1. Identificació de la seqüència:
Primer() =
Següent () =
FinalSeq () =
- 2. Identificació de l'esquema:

Recorreguts de seqüències amb el nombre d'elements conegut.

- Els problemes que es poden reduir a seqüències tals que es controla el seu final amb el nombre d'elements i que, a més a més, s'ha d'aplicar l'esquema de recorregut, es poden solucionar amb la instrucció especialitzada `for`.
- La variable que controla el nombre d'elements de la seqüència s'anomena comptador.

2.4.4 Esquema de recorregut

39

Recorreguts de seqüències amb el nombre d'elements conegut. **For**

```
<inicialitzacions>
for(idx = <inici>; idx< = <final>; idx = Incrementa(idx)) {
    <tractar_element>
}
<finalitzacions>
```

```
for (int num = 0; num <= 10; num++) {
    printf("5 x %d = %", num, (5 * num));
}
```

- Si cal fer el bucle en ordre decreixent: la condició seria l'oposada i en lloc d'incrementar, la decrementaríem.

2.4.4 Esquema de recorregut

40

Recorreguts de seqüències amb el nombre d'elements conegut.

While

```
<inicialitzacions>
idx = <inici>;
while (idx <= <final>) {
    <tractar_element>
    idx = Incrementa(idx);
}
<finalitzacions>
```

```
int num;
num = 0;
while (num <= 10) {
    printf("5 x %d = %", num, (5 *
num) ) ;
    num = num + 1;
}
```

Contingut

2.4 Estructures de control: iteratives (avançat)

2.4.1 Introducció

2.4.2 Concepte de seqüència

2.4.3 Esquemes de programació de seqüències

2.4.4 Esquema de recorregut

2.4.5 Esquema de cerca

2.4.6 Recomanacions

2.4.5 Esquema de cerca

42

Objectiu: Donada una seqüència d'elements es vol accedir a **un element que compleix** una determinada propietat (o condició booleana). Apliqueu l'esquema que vulgueu dels 2 següents:

Esquema 1:

```
<inicialitzacions>
condicio_cerca = false
elem = Primer();
while ( !FinalSeq(elem) && !condició_cerca ) {
    elem=Següent(elem);
}
<condicio_cerca = !FinalSeq(elem)>
<finalitzacions>
```

Esquema 2: Permet no avaluar el sentinella

```
<inicialitzacions>
elem = Primer();
trobat = false;
while ( !FinalSeq(elem) && !trobat )
{
    if (condició_cerca) {
        trobat = true;
    } else {
        elem=Següent(elem);
    }
}
<finalitzacions>
```

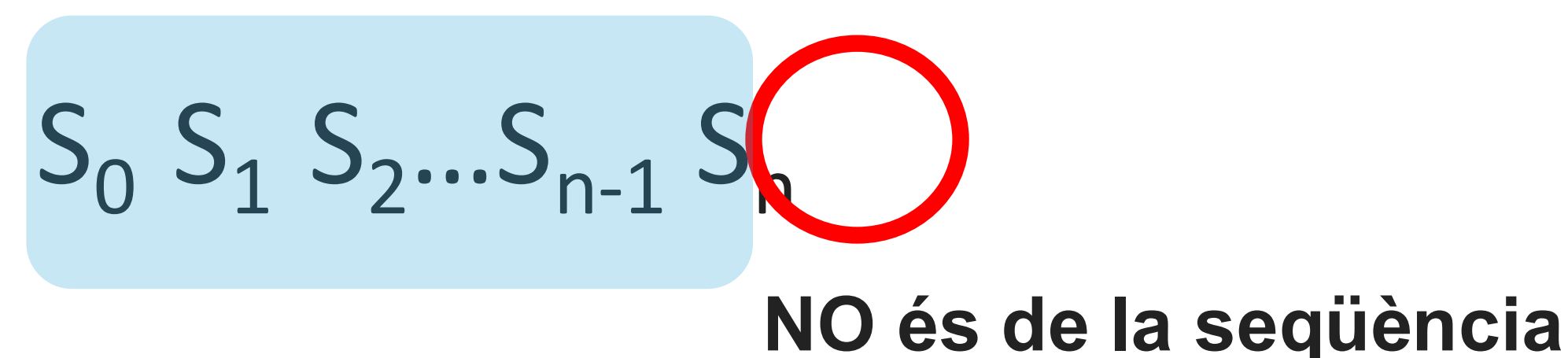
2.4.5 Esquema de cerca

43

L'esquema de cerca garanteix que:

- Cada element de la seqüència només es tracta una vegada.
- Sempre es tracta l'element actual primer i després avança al següent element de la seqüència.
- No es tracta cap element que no sigui de la seqüència.
- A cada iteració tot element de la part de l'esquerra de la seqüència no compleix la propietat de cerca.
- En acabar, s'ha comprovat que la condició de cerca no es compleix per a tots els elements de la seqüència o bé s'ha trobat el primer element de la seqüència que compleix la propietat de cerca.
- S'ha de garantir que la propietat de cerca es pot avaluar sobre el sentinella de la seqüència (Esquema 1)

Seqüència de n elements:



Seqüència fins centinella:



Quina és la seqüència?

1. Identificació de la seqüència:
2. Primer ():
3. Següent (x):
4. FinalSeq (x):
5. Identificació de l'esquema (recorregut o cerca, amb cerca indicar condició):

PLANTILLA A EMPLENAR PER A
IDENTIFICAR LA **SEQÜÈNCIA** I
L'**ESQUEMA** (RECORREGUT O
CERCA)

Quina és la seqüència i quin és l'esquema?

- **Exemple: Donada una seqüència d'enters introduïts per teclat acabada en 0, esbrinar si algun d'aquests enters és la CLAU SECRETA d'accés a un compte.**

1. Identificació de la seqüència:
2. Primer ():
3. Següent (x):
4. FinalSeq (x):
5. Identificació de l'esquema (recorregut o cerca, en cas de cerca indicar la condició):

Esquema 2

```
#include <stdio.h>
#define CLAU_SECRETA 725
int main(void) {
    int clau; // clau entrada per l'usuari
    int atura; // indica si s'ha encertat la clau (0 = false, 1 = true)
    printf("Clau secreta?\n");
    scanf("%d", &clau);
    atura = 0;
    while ((clau != 0) && !atura) {
        if (clau == CLAU_SECRETA) {
            atura = 1;
        }
        else {
            printf("Clau secreta?\n");
            scanf("%d", &clau);
        }
    }
    if (atura) {
        printf("Acces obert\n");
    } else {
        printf("No ho has encertat!\n");
    }
    return 0;
}
```

```
<inicialitzacions>
elem = Primer();
trobat = false;
while ( !FinalSeq(elem) && !trobat ) {
    if (condició_cerca) {
        trobat = true;
    } else {
        elem=Següent(elem);
    }
}
<finalitzacions>
```

```
/* Llistat de les taules de multiplicar del 1 al 10 */
/* Identificació de la seqüència: enters 1 al 10
Primer: 1
Seguent(base): base + 1
FinalSeq(base): base > 10
Identificació de l'esquema: Recorregut
*/
for (int base = 1; (base <= 10) ; base++) {
    printf("Taula del %d\n", base);
    /* Identificació de la seqüència: enters del 0 al 10
    Primer: 0
    Seguent(num): num + 1
    FinalSeq(num): num > 10
    Identificació de l'esquema: Recorregut */
    for (int num = 0; (num <= 10) ; num++) {
        printf("%d x %d = %", base, num, (base * num));
    }
}
```

Contingut

2.4 Estructures de control: iteratives (avançat)

2.4.1 Introducció

2.4.2 Concepte de seqüència

2.4.3 Esquemes de programació de seqüències

2.4.4 Esquema de recorregut

2.4.5 Esquema de cerca

2.4.6 Recomanacions

2.4.6 Recomanacions

Sentències `continue` i `break`

Permeten forçar la sortida d'un bucle

- Amb `continue`, s'acaba la iteració actual i es torna a executar la següent iteració.
- Amb `break`, s'acaba completament la execució de la iterativa (o del `switch`).
- NO es deu abusar de `break`, fent coses com:

```
num = 1;
while (true) {
    printf(" "+num+" ");
    num = num + 1;
    if (num > 100)
        break;
}
```



- **FIXEU-VOS** que els esquemes es basen en només `while` i `for`. **SENSE** `continue` o `break`.
- Per afavorir la creació d'un codi estructurat, **només s'ha d'utilitzar** `continue` i `break` **en casos excepcionals** en els que hi ha una millora en la legibilitat del codi.

Ús de if anidats vs ús de continue (exemple d'un cas extrem)

```
while (condicio1) {  
    metode1();  
    metode2();  
    if (condicio2) {  
        metode3();  
        metode4();  
        if (condicio3) {  
            metode5();  
        }  
    }  
}
```



```
while (condicio1) {  
    metode1();  
    metode2();  
    if (!condicio2) {  
        continue;  
    }  
    metode3();  
    metode4();  
    if (!condicio3) {  
        continue;  
    }  
    metode5();  
}
```



Exercicis

1. Dissenyar un programa que donat un enter pel teclat, n , compti el nombre de dígit que el componen.
2. Donada una seqüència d'enters positius entrats per teclat acabada en -1, calcular la seva mitjana.
3. Donada una seqüència d'enters positius entrats per teclat acabada en -1, calcular la mitjana dels nombres de la seqüència que són entre 1 i 100. Suposem, llavors, que en aquesta seqüència pot haver-hi enters fora de l'interval esmentat.
4. Feu un programa que mostri per pantalla els múltiples de 13 positius inferiors a 1000.
5. Feu un programa que mostri per pantalla el 1000 primers múltiples de 13 positius.
6. Donada una seqüència d'enters pel teclat acabada en 0, feu un programa que esbrini si tots són negatius.
7. Donada una seqüència d'enters positius pel teclat acabada en -1, feu un programa que trobi el màxim. Com modificaries aquest programa per a que trobi el mínim també?
8. Donada la seqüència dels múltiples de 7 menors de 10000, comptar el nombre de vegades que surt el dígit 2.